

Designing Autonomous Vehicles in Simulated Environments

Contents

Abstract.....	1
Background Research.....	1
Objective	3
Environment	3
Agents	4
Agent Design	4
PPO Model	5
GAIL Model.....	6
GAIL + PPO Model	8
Further Inspection.....	10
Improvements and proposals	11
Conclusion.....	11
Appendix	11
Assets	12

Abstract

Human error is estimated to be the highest contributing factor to driving and road accidents [1], and with the continuous breakthroughs in the field of machine learning and AI, autonomous vehicles are thought to be the solution to this problem. An autonomous vehicle is a vehicle that is fully driven and operated by itself or, specifically, by an AI. There have been vehicles with autonomous features in the market starting the 1980s. More recently, Tesla's Model S has been the most successful, and the only car manufacture that claims to have "Full self-driving capabilities" [7]. While that term might be technically incorrect, the Tesla is the closest we have to a full autonomous vehicle right now.

Background Research

One of the challenges that could face autonomous vehicle development is training and testing, as some extreme scenarios are too dangerous and expensive to test in the real world. Hence, a huge part of training and testing AVs is dependant on software that simulates the real world [2]. As of 2019, Waymo has logged 15 billion miles in testing through computer simulations.

There are 2 distinct types of traffic simulators. One is large traffic simulators where the simulators are used to simulate traffic flow of road networks including a lot of vehicles. For this kind of simulators,

everything is simplified, and car behaviour is sometimes even reduced to discrete behaviour. For instance, a car can be in either lane a or lane b, with no transition in between. This type of simulators is mainly used for large traffic flow over complex road structures rather than accurate singular car behaviour. The other type of simulators is small scale robotic simulators, in this case, they are used to simulate the behaviour of a small number of vehicles (or even one) as accurately as possible. Ideally, we want a simulator to be as close to the real world as computationally possible. In the real world, a car knows about its surrounding environment through multiple different sensors that detect different types of changes. For instance, a GPS receiver is used to determine a car's current location, time, and velocity. Luminosity sensors are used to detect amount of surrounding light. Infra-red cameras are used to view the surrounding world in an infra-red spectrum. Laser scanners are used to measure distance, size, and shape of surrounding objects. Ultrasound sensors are, also, used to measure distances and shapes of close-range objects. All these sensors and types of data inputted to the car's system are used by the car to comprehend its environment and act accordingly.

A good simulator would ideally have implementations for all or most of those sensors. Additionally, to further imitate the real world, the simulator would imitate failures of such sensors and noise data. Besides the sensors, there are some features that are essential for the functionality of the simulator. For instance, collision detection, the ability to detect when a vehicle collides with another vehicle or an obstacle. According to [2], the following list of requirements are essential for simulators: Perception, multi-view geometry, path planning, vehicle control, 3D virtual environment, traffic infrastructure, traffic scenarios simulation, 2D/3D ground truth.

There are multiple different simulators available with different features and complexity. There are some that are used for simple scenarios like Simulink. Some simulators use a manual process of creating 3D models and physics definitions, like Gazebo. Additionally, CarSim is made especially for vehicle simulations containing a library of vehicles parameters. Furthermore, some game engines are also used for vehicle simulation like Unreal Engine and Unity. In some cases, even games, like GTA V, are used as simulators for autonomous vehicles.

Driving a car is a high complexity task. As a result, to build an autonomous vehicle, manufacturers usually use multiple complex models for different sub tasks. There are 3 main sub-tasks, namely perception planning and control. Perception is how the car perceives its surrounding world, and as previously mentioned, that includes collection of data through sensors and filtering them. For planning, an AV's job usually consists of path planning and mission planning. This is where the vehicle creates a path to reach its goal depending on the data collected. This also includes decision making models and collision avoidance. Lastly, control is where the vehicle actually takes actions such as steering, accelerating, braking, etc.

One of the ways to develop and train autonomous vehicles in terms of Control is reinforcement learning. RL is an unsupervised ML algorithm where an agent tries to maximise a reward function. The agent starts by doing random actions, and different actions have different rewards. The agent then learns to do the actions that maximises his reward and steer away from the actions that minimise that reward function. One way to model that is through a Markov Decision Process, where an MDP is represented as a 6-tuple (S, A, T, γ, D, R) . S is a finite set of states representing the environment. A is a finite set of actions the agent can take at a certain state, T is the state transition probability matrix that is the probability of the system transitioning between every pair of state. $\gamma \in [0, 1]$ is the discount rate that guarantees convergence. D is the initial state distribution. R is the reward function. The goal of an MDP is to find a policy π for an agent where it specifies the action to take at the current state $S(t)$. The optimal Policy π^* maximises the cumulative discounter reward over an infinite horizon.

Different ML methods could be used to solve reinforcement learning problems such as ANNs, naïve Bayes, Gaussian processes, and SVMs.

One of the main challenges of RL is to create a good/optimal reward function. Added to that is the complexity of the task at hand. As a result, inverse reinforcement learning could also be used to solve the problem. IRL is somewhat similar to RL. In IRL, the agent is provided with training data from an expert (ie a human driver) that is trying to solve the problem (reach the goal). The goal of IRL is to use that training data to find a reward function from the observed behaviour.

Another paradigm for autonomous vehicles is end-to-end AVs [4]. This is where the AV is designed as a single distinct responsible to map output actions from input signals. End-to-end systems are computationally more efficient as they are self-optimising. However, this comes at the cost of verifiability as it works as a black box. Once again, this type of AVs can be developed using both reinforcement learning and imitation learning algorithms. Imitation learning algorithms include behavioural cloning, direct policy learning, Generative Adversarial Imitation Learning (GAIL) and IRL.

BC is where the agent just learns to copy the behaviour of the training data as close as possible. With this occurs the problem of weak generalisability as it is limited by the cases that are provided in the training dataset, and hence requires a large training dataset. Direct Policy learning tries to improve upon the limitations of BC by providing an expert at training time that the agent querying the expert at run-time. This is usually when the agent is presented with an underrepresented scenario in the original dataset. GAIL is similar to IRL, with the difference being that in GAIL, the agent is learning the policy rather than the reward function. This is done by training a discriminator to identify whether a specific instance is learnt or given by an expert [3].

Previous work was done [5] in comparing 2 reinforcement learning algorithms namely PPO and SAC in the framework of autonomous vehicles. PPO is an on-policy approach that learns from data that it collects from the current state of its environment. Whereas SAC is an off-policy algorithm that depend on a buffer stored data from previous episodes. It is worth noting that this work was achieved using the same framework and simulator discussed in this project.

Objective

Given the previous research done in this field, we try to compare reinforcement learning methods against imitation learning methods to determine which, if any, is better at solving the task. The task at hand being an end-to-end autonomous vehicle that is trying to reach a nearby goal while avoiding collision with obstacles. This can be determined by the success rate of both methods, and the average speed of which the task is performed.

We use PPO as the RL algorithm and GAIL as the IML algorithm. Both algorithms are provided within a framework named “ml-agents” that is available through the Unity game engine.

Environment

The environment in which we will design, train, and test our agents will be developed using Unity’s game engine. For the scope of this project, the environment developed will be somewhat a simpler model compared to what a car manufacturer would use. To elaborate, we will simplify how the agent would view and interact with its environment. Ideally, this would be designed to imitate a real car in the street where all its data is gained through the various sensors that it uses. Such data would then be analysed using the different machine learning models to try and form an understanding or a

representation of the surrounding environment. This is slightly more complex than the scope of this project.

The environment where the agent would be trained and tested is chosen to be a square piece of land with buildings on the outskirts and a cube that is randomly located within the piece of land that represents the destination for our agent. The area of the environment is decided to be approximately 100 metres squared. This represents a medium-sized environment for our agent. That is because if the area is too large, training the agent becomes much harder and requires more time. On the contrary, if the area is too small, the agent just stumbles across the destination and training could be trivial or inaccurate.

For visualisation, the plane/floor changes colour according to the agent's performance. To elaborate, when the agent collides with one of the buildings, we view that as a failed trip. When the agent reaches the destination cube, we view that as a successful trip. Accordingly, upon the completion of a successful trip the ground changes colour to green, and upon the completion of an unsuccessful trip the ground changes colour to red. By default, on the first trip the ground colour is grey. We further discuss the use of such visualisation later in the training section.

Agents

Agent Design

The generic agent is designed as a car downloaded from the Unity asset store. 4 wheel colliders were added to the car object. To try and imitate realistic behaviour, the acceleration force, brake force, wheel turning angle, car weight, friction parameters were optimised through trial and error. The agent perceives its environment through different methods. The `AddObservation()` method from within the agent CS file was used to pass information to the agent in the form of observations. The passed information were the agent's position and the destination cube's position. However, the position in Unity is a 3-dimensional position, so essentially, we are passing 3 observations for every position given, each for the x, y, and z values. Additionally, we give the agent the distance between it and the destination as a float. These decisions are to somewhat imitate the job of a GPS where it would be able to reason about the distances and the location of itself and its surroundings.

In addition, the agent has 10 RaySensors coming out of it with view of 5 metres. This is to imitate the various sensors a real autonomous vehicle would use such as laser sensors. RaySensors can detect a goal or a building if it collides with it and feed such information to the agent as observations. This means that our agent is being fed 17 observations in total between the position data and the RaySensors data.

For output, the agent has 2 continuous variables with variables between -1 and 1 and one discrete variable with values 0 and 1. The 2 continuous variables being the gas pedal and the steering wheel. The higher the positive value for the gas-pedal the more the acceleration. Whereas negative values represent acceleration in the reverse direction. The discrete value represents whether or not the brakes are pressed.

As for the reward function, it is similar across both models. Even though GAIL does not use extrinsic rewards ie. The reward function setup does not affect it, the reward function is still used to measure performance. The reward function consists of adding a reward of -5 when hitting a building and a reward of +10 when reaching a goal. Additionally, through out every training episode, the agent is given a slight negative reward on every turn of the wheel and on every reverse movement. The maximum penalty the agent could be given is -2 for every episode. This happens in the case that

throughout the whole episode, the agent was steering the wheel and driving backwards. Such penalty is added to encourage moving forward while minimising rotation so that the agent could hopefully take the shortest route to reach the goal.

Training the agent is done through the ml-agents framework where parameters of a DNN are passed through a configuration file, and every training instance is given a run id. For both our PPO and GAIL models, training is done for 15 million steps, where every episode takes a maximum of 5000 steps. To further speed up training, 25 instances of the agent and the environment are created and run simultaneously, every episode every agent takes updates and uses the same Neural Network.

PPO Model

For the PPO model, the following parameters are chosen.

```
behaviors:
  DriveCar:
    trainer_type: ppo
    hyperparameters:
      batch_size: 2560
      buffer_size: 10240
      learning_rate: 0.0003
      beta: 0.09
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 5
      learning_rate_schedule: linear
      beta_schedule: linear
      epsilon_schedule: linear
    network_settings:
      normalize: false
      hidden_units: 256
      num_layers: 2
      vis_encode_type: simple
      memory: null
      goal_conditioning_type: hyper
      deterministic: false
    reward_signals:
      extrinsic:
        gamma: 0.85
        strength: 1.0
        network_settings:
          normalize: false
          hidden_units: 128
          num_layers: 2
          vis_encode_type: simple
          memory: null
          goal_conditioning_type: hyper
          deterministic: false
      curiosity:
        strength: 0.3
        gamma: 0.99
        learning_rate: 0.0001
    init_path: null
    keep_checkpoints: 5
```

Training the PPO Model for 15 million steps took approximately 8 hours 12 minutes and gave the following results.



Upon inspecting the cumulative reward graph, we notice that the reward value kept fluctuating throughout the training period without convergence. The value at every point shows the mean reward for 5000 steps. The graph peaked at about -1.1 mean reward and had a trough of about -3.2 mean reward. However, towards the final 2 million steps the mean reward averages about -1.4. From this data we conclude that the model has not been successful at learning the task at hand of reaching the goal. Furthermore, the episode length graph shows an interesting trend as the average episode length starts increasing until it reaches the max steps at around 11 million steps. Based on that data we can deduct that the agent has learnt to avoid hitting the building and thus ending the episode early.

To analyse the model's final behaviour, we attach the resulted .onnx file to an agent and inspect the behaviour for 10 episodes. We notice that the agent does not hit a building in any of the 10 episodes with a 100% success rate in collision avoidance. However, we also notice that the agent is randomly accelerating in reverse in circular motion, only arriving to its target 3 out of the 10 episodes in what seems to be random lucky fashion rather than a goal oriented one. Hence, we conclude that the PPO agent has failed to learn the main task at hand but has learned to avoid hitting buildings.

GAIL Model

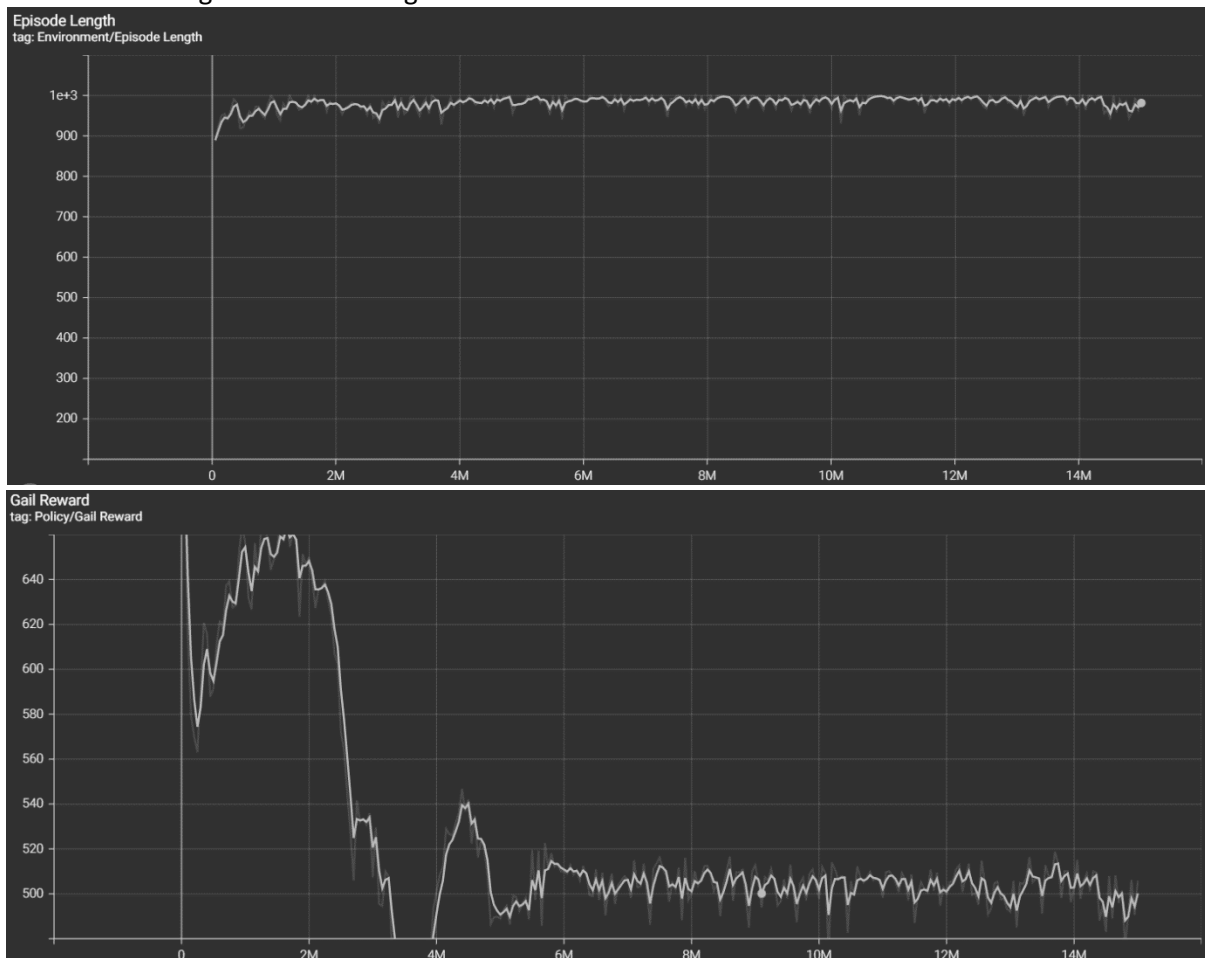
For the GAIL model, the following parameters are chosen.

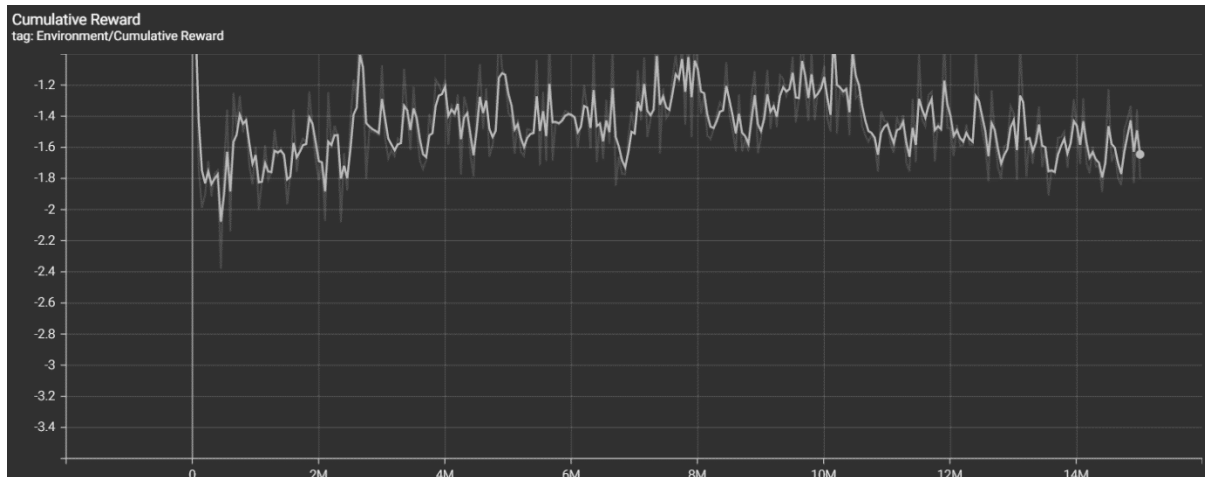
```

behaviors:
  DriveCar:
    trainer_type: ppo
    hyperparameters:
      batch_size: 2560
      buffer_size: 10240
      learning_rate: 0.0003
      beta: 0.09
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 5
      learning_rate_schedule: linear
      beta_schedule: linear
      epsilon_schedule: linear
    network_settings:
      normalize: false
      hidden_units: 256
      num_layers: 2
      vis_encode_type: simple
      memory: null
      goal_conditioning_type: hyper
      deterministic: false
    reward_signals:
      gall:
        strength: 1.0
        gamma: 0.99
        demo_path: D:\CompSci\Machine Learning\Machine Learning Unity\Machine Learning\Assets\Demonstrations\DriveCarDemo2.demo
        learning_rate: 0.0003
        use_actions: true
        use_vail: true
      curiosity:
        strength: 0.1
        gamma: 0.99
        learning_rate: 0.0001

```

Unlike PPO, a GAIL model requires expert training data to try and extract a policy from it and use it. The training set that was provided for the GAIL model as expert data was constructed by recording the car being driven by a human using the keyboard. The dataset consisted of 98 episodes and approximately 13000 steps. Training the GAIL Model for 15 million steps took approximately 9 hours 30 minutes and gave the following results.





Upon inspecting the previous graphs and comparing the values with the PPO model, we find that the GAIL model had learnt not to collide with building much quicker, almost from the beginning. This is shown from the episode length graph as almost all episodes from the start were of max step number of steps. However, the GAIL model still has a mean cumulative of around -1.5 as its final behaviour. That means that it has failed to learn to reach the destination too, similar to the PPO model.

We analyse the GAIL model final behaviour in a similar manner to how we analysed the PPO model. We attach the .onnx brain to the agent and inspect it's behaviour for 10 episodes. Once again, we notice that the agent does not collide with any of the buildings throughout all 10 episodes. However, the agent reaches its destination 0 times out of 10. Upon further inspection, we notice that the agent almost doesn't change its position but only rotates in place in a rough, rocky manner. We also notice that the agent rotates in the direction of the cube 7 times out of 10. However, it continues rotating and does not stop when its rotation aligns with the cube. Such results are not strong evidence that the agent learnt to rotate in the right direction. It does, however, show that the agent learnt the rough rotation in place from the training dataset.

As both the GAIL and the PPO models gave disappointing results, we try to combine both methods and further optimise the training parameters for better results.

GAIL + PPO Model

For the GAIL + PPO Model, the following parameters are chosen.

```

behaviors:
  DriveCar:
    trainer_type: ppo
    hyperparameters:
      batch_size: 2560
      buffer_size: 10240
      learning_rate: 0.0003
      beta: 0.09
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 5
      learning_rate_schedule: linear
      beta_schedule: linear
      epsilon_schedule: linear
    network_settings:
      normalize: false
      hidden_units: 256
      num_layers: 4
      vis_encode_type: simple
      memory: null
      goal_conditioning_type: hyper
      deterministic: false
    reward_signals:
      extrinsic:
        strength: 0.9
        gamma: 0.99
      gail:
        strength: 7.0
        gamma: 0.99
        demo_path: D:\CompSci\Machine Learning\Machine Learning Unity\Machine Learning\Assets\Demonstrations\DriveCarDemo3.demo
        learning_rate: 0.0003
        use_actions: false
        use_vail: true
      curiosity:
        strength: 0.1
        gamma: 0.99
        learning_rate: 0.0001

```

In a trial to improve the performance of the model, we double the training time to 30 million steps. Training took around 16 hours 18 minutes. Furthermore, we use more training data for GAIL. The training dataset for this model contains xxx instances with xyz steps. Not only that, but to counter the rocky behaviour found in the GAIL model while rotating, this time training is done using a — controller instead of a keyboard. This is because for a keyboard a press of a key has a value of one which represents a full pedal flooring or rotating a steering wheel maximally to either side. Hence,

training using a controller is used to give a smoother driving experience. The following results were given after training and compared to the previous models.



Upon inspecting the graphs, we notice that the GAIL + PPO model almost does not differ than the GAIL model at all, even with double the training time and theoretically better training data. Procedurally, we attach the .onnx brain to an agent to inspect the final behaviour of the model. After 10 episodes, we notice that the behaviour of the model is very similar to the GAIL model behaviour. The agent does not collide with any buildings, but it continues to rotate in place. However, the rotation is now noticeably smoother than the GAIL model. That is probably the case due to using a controller for recording the training dataset. For the GAIL + PPO Model, the agent rotates in the direction of the destination 5 times out of 10 which is still not enough evidence that the agent has learnt to rotate towards its goal.

Further Inspection

To Inspect the three different models, run the unity project, and run the scene. Pressing the numbers 1 2 3 and 4 on the keyboard will toggle the different cameras for the different models.

Improvements and proposals

There are multiple factors that could be addressed to improve upon this project both from a ML perspective and from an environmental setup perspective. To elaborate, one of the biggest factors that affected the performance of all the models are believed to be the reward functions. Perhaps it would have been better to give positive reward amongst moving forward rather than a negative reward amongst moving backwards. Additionally, a system similar to a checkpoint system would have made the task more learnable. Such system would make the agent able to learn to get closer to the destination, rather than hoping that the agent will reach the correlation between the distance observation and the final reward by itself. Similarly, there exists episodes where the agent just stumbles upon the destination, which would reinforce a certain behaviour due to the relatively weak reward function.

Another thing that needs to be improved and is probably the main reason the models performed badly is not using hyper-parameter tuning methods. Methods such as grid search, simulated annealing, bandit-optimisation, and Bayesian optimisation methods [6] could be used to tune the hyper parameters such as learning rate, epsilon and curiosity to allow the agent to learn the task at hand optimally. Additionally, experimenting and trying different number of hidden layers and number of neurons per layer.

Conclusion

In this project we tried to use unsupervised learning methods such as PPO for reinforcement learning, GAIL for imitation learning, and a combination of both models to train an end-to-end agent to drive a car reaching a destination while avoiding collisions. While all 3 models technically failed to learn the task at hand optimally, different interesting behaviours were shown.

Appendix

The final results are in the repository environment/Scripts/results. For the PPO model, DrivePPO6 was the final model trained and used.

- [1] M. C. Figueiredo, R. J. F. Rossetti, R. A. M. Braga and L. P. Reis, "An approach to simulate autonomous vehicles in urban traffic scenarios," 2009 12th International IEEE Conference on Intelligent Transportation Systems, 2009, pp. 1-6, doi: 10.1109/ITSC.2009.5309524.
- [2] P. Kaur, S. Taghavi, Z. Tian and W. Shi, "A Survey on Simulators for Testing Self-Driving Cars," in 2021 Fourth International Conference on Connected and Autonomous Driving (MetroCAD), Detroit, MI, USA, 2021 pp. 62-70.
- [3] L. Le Mero, D. Yi, M. Dianati and A. Mouzakitis, "A Survey on Imitation Learning Techniques for End-to-End Autonomous Vehicles," in IEEE Transactions on Intelligent Transportation Systems, doi: 10.1109/TITS.2022.3144867.
- [4] You, Changxi, Jianbo Lu, Dimitar Filev and Panagiotis Tsotras. "Advanced planning for autonomous vehicles using reinforcement learning and deep inverse reinforcement learning." Robotics Auton. Syst. 114 (2019): 1-18.
- [5] A. J. M. Muzahid, S. F. Kamarulzaman and M. A. Rahman, "Comparison of PPO and SAC Algorithms Towards Decision Making Strategies for Collision Avoidance Among Multiple Autonomous Vehicles," 2021 International Conference on Software Engineering & Computer Systems and 4th International Conference on Computational Science and Information Management

[6] Fridman, L., Terwilliger, J., and Jenik, B., “DeepTraffic: Crowdsourced Hyperparameter Tuning of Deep Reinforcement Learning Systems for Multi-Agent Dense Traffic Navigation”

[7] https://www.tesla.com/en_gb/models

Assets

<https://assetstore.unity.com/packages/3d/environments/urban/cartoon-low-poly-city-pack-lite-166617>

<https://assetstore.unity.com/packages/3d/vehicles/land/low-poly-civilian-vehicle-5-124987>